Reflexión del equipo: despliegue automatizado

Proyecto: FarmToTable · **Actividad:** Despliegue automatizado con GitHub Actions · **Equipo:** FarmToTable

La actividad nos permitió organizar el despliegue de FarmToTable como un proceso verificable. Separamos los nueve microservicios en repositorios independientes y creamos un décimo repositorio, pipelines, para compartir la automatización. Comprendimos que separar el código exige revisar sus dependencias: cada servicio debía poder instalarse, comprobarse y construirse sin depender de archivos externos de la aplicación original.

La principal decisión fue concentrar la lógica común en un workflow reutilizable mediante workflow_call. Cada repositorio invoca ese flujo y proporciona su configuración de destino. Esta estructura reduce la duplicación y permite mantener los mismos controles para todos los microservicios. También identificamos una responsabilidad: los cambios en el repositorio central deben revisarse cuidadosamente porque pueden afectar los despliegues de todo el proyecto.

El pipeline sigue la secuencia test → docker → deploy. Primero ejecuta instalación de dependencias, generación de Prisma, comprobación de tipos, análisis de código, compilación y pruebas. Solo después construye y publica la imagen Docker, identificada con el SHA del commit. Finalmente envía el mismo código probado a Railway y comprueba el resultado del despliegue. Railway reconstruye la imagen; desplegar directamente el artefacto de GHCR queda como mejora para evitar esa segunda construcción.

Para comprobar el bloqueo, ejecutamos una prueba intencionalmente incorrecta en cada uno de los nueve microservicios. En todos los casos falló el trabajo de pruebas y quedaron omitidos Docker y deploy. Después lanzamos una ejecución correcta en cada repositorio: los nueve completaron las tres etapas. Guardamos dieciocho capturas y los enlaces de ambas ejecuciones. La comprobación final del gateway mostró los nueve servicios disponibles con HTTP 200. Esta evidencia demuestra la automatización y su control de errores, aunque no reemplaza las pruebas funcionales completas del negocio.

Durante la conexión encontramos dos dificultades: una migración de Producer intentaba crear objetos que ya existían y algunos servicios utilizaban rutas de salud diferentes. Contrastamos el esquema antes de reconciliar el historial de migraciones y configuramos la ruta correspondiente a cada servicio. Aprendimos que una compilación exitosa no garantiza que la aplicación esté preparada para recibir solicitudes.

El manejo de credenciales fue otro aprendizaje central. El token de Railway se guardó cifrado en GitHub Actions Secrets y limitado al proyecto y entorno de producción; las credenciales de bases de datos y las claves de aplicación permanecieron en Railway. Excluimos archivos de entorno y claves privadas de los repositorios. Como equipo concluimos que automatizar mejora la repetibilidad y la trazabilidad, pero requiere controles y revisión. Proponemos fortalecer la protección de ramas, incorporar aprobación independiente de producción y ampliar las pruebas de integración como siguientes mejoras.